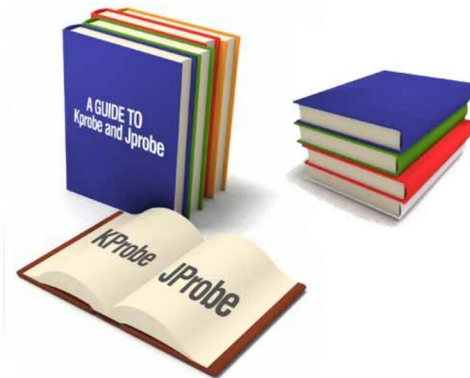




Kernel Debugging Using KPROBE and JPROBE

Debugging is like plumbing; it involves fixing difficult, hidden problems—so, besides the vital experience, both debuggers and plumbers must have a specialised set of tools at hand for each task. Targeted at Linux kernel newbies and kernel debuggers, this article introduces two tools, Kprobe and Jprobe, which are simple, and get the work done.

First, we must pay homage to the best-known kernel-debugging technique/tool, the *'printk'*.

It is a universal tool, customisable enough to provide you the desired output—and usually when all else fails, this can save the day! It's very easy to use; it simply prints whatever you like, to track the execution of a module by printing its status after each step, and thus pin-pointing bugs and defects.

Every good thing comes at a price, however—and *printk's* price is its static nature, which may become a bit of a problem when frequent changes are

required in the information needed to be printed while debugging. Also, in order to implement *printk*, you need to recompile the kernel with *printk* statements added to the specific module you're debugging. This is usually a time-consuming process of building, installing and rebooting the kernel.

These problems can be resolved by using the dynamic tools *Kprobe* and *Jprobe*. The great thing about Linux is that kernel 2.6.x already contains *Kprobe*; we don't need installation or patching to try it. So it's time for some hands-on fun!